

Topic 4: Back-end Development

Welcome to back-end development! This is where we build the engine of a web application. The back-end is responsible for everything the user doesn't see: managing data, processing requests, and ensuring security. In this topic, you'll learn how to build a server with Node.js, connect it to a database, and secure it with modern authentication techniques.

Objectives and Key Results

Objective	Key Result
Understand the client-server architecture.	You can explain the roles of a client and a server, describe the request-response cycle, and use Postman to send test requests to a server.
Build a functional Node.js application.	You can set up an Express server, define routes to handle different requests, and connect the server to a PostgreSQL database using node-postgres.
Master API development with HTTP and CRUD.	You can map HTTP methods (GET, POST, PUT, DELETE) to CRUD operations and create API endpoints that perform these operations on your database.
Implement user authentication and authorization.	You can explain the purpose of authentication, implement a JWT-based login system, and create protected routes that only authenticated users can access.

4.1 Client vs. Server

Every web application operates on a **client-server model**. Think of it like a restaurant.

- **The Client:** This is you, the customer sitting at the table. You don't go into the kitchen yourself; instead, you make a **request** to the waiter for a specific dish. In the web world, the client is typically a web browser (like Chrome or Firefox) or a mobile app. It's responsible for the user interface (what you see and interact with).
- **The Server:** This is the restaurant's kitchen. It receives your order (the request), prepares the dish, and sends it back out with the waiter. A server is a powerful computer that "listens" for requests from clients. When a request arrives, the server performs the necessary logic—like fetching data from a database, running calculations, or talking to other services—and sends back a **response**.

This communication happens through the **request-response cycle**. A client sends an **HTTP request** to a server's specific URL. The server processes it and sends back an **HTTP response**, which might contain data (often in JSON format), a full HTML page, or just a status code.

Postman: Your API Sidekick

When building a back-end, you often don't have a front-end client ready for testing. This is where tools like Postman come in. Postman is an API platform that lets you create and send any kind of HTTP request to your server, allowing you to test your back-end logic without needing a browser.

Resources

- **Client-Server Model Overview:** A simple explanation of the core concept.
- **Postman Learning Center:** The official documentation and tutorials for Postman.

4.2 Node.js and Express: The Back-end Foundation

Node.js is a runtime environment that lets you run JavaScript code on the server, outside of a web browser. This unlocks powerful capabilities like accessing the file system, managing databases, and handling network requests directly. With Node.js, you can use one language—JavaScript—for both your front-end and back-end code.

Node.js comes with **npm (Node Package Manager)**, a massive registry of open-source libraries (packages) you can add to your project. Two of the most essential packages for back-end development are Express and node-postgres.

Express.js

Express is a fast, minimalist, and flexible web framework for Node.js. It simplifies the process of building a server and handling requests. Its two core features are:

1. **Routing:** Routing is how an application responds to a client request to a particular endpoint, which consists of a URL (or path) and a specific HTTP request method (e.g., GET, POST). Express provides a simple way to map these endpoints to handler functions.

```
// A simple Express route
app.get('/users', (req, res) => {
  // This is the handler function.
  // It runs when a GET request is made to '/users'.
  res.send('List of all users');
});
```

2. **Middleware:** Middleware functions are functions that have access to the request object

(req), the response object (res), and the next function in the application's request-response cycle. They can execute code, make changes to the request and response objects, end the request-response cycle, or call the next middleware in the stack. Middleware is perfect for tasks like logging requests, validating user input, or checking for authentication.

node-postgres (pg)

To connect your Node.js application to a PostgreSQL database, you need a driver.

node-postgres (often just called **pg**) is the most popular and robust library for this purpose. It allows you to create a connection "pool" to your database and execute raw SQL queries or use prepared statements to prevent security vulnerabilities like SQL injection.

Resources

- **Official Node.js Guides:** Learn the fundamentals of Node.js.
- **Express "Hello World" Example:** Your first steps with the Express framework.
- **node-postgres Documentation:** The complete guide to connecting Node.js with PostgreSQL.

4.3 HTTP Methods and CRUD

The **Hypertext Transfer Protocol (HTTP)** defines a set of **request methods** that indicate the desired action to be performed for a given resource. These methods map directly to the CRUD operations we learned about in the previous topic.

- **GET (Read):** Used to **retrieve** data from the server. A GET request to /users would fetch a list of users, while a GET to /users/123 would fetch the specific user with ID 123. This is a read-only operation.
- **POST (Create):** Used to **submit** new data to the server, resulting in the creation of a new resource. For example, a POST request to /users would carry the data for a new user in its request body and create that user in the database.
- **PUT (Update):** Used to **update** an existing resource. The request body contains the complete new version of the resource you are updating. For example, a PUT request to /users/123 would replace the entire user record for user 123 with the new data.
- **DELETE (Delete):** Used to **remove** a specific resource from the server. A DELETE request to /users/123 would delete the user with ID 123.

By creating routes for each of these HTTP methods, you build a **RESTful API**—a standard and intuitive way for clients to interact with your server's data.

Resources

- **MDN: HTTP request methods:** A comprehensive overview of all HTTP methods.

4.4 Authentication with JWT

Authentication is the process of verifying who a user is. **Authorization** is the process of determining what an authenticated user is allowed to do. You can't have authorization without authentication.

For modern web applications, one of the most popular methods for authentication is using **JSON Web Tokens (JWT)**. A JWT is a compact and self-contained way for securely transmitting information between parties as a JSON object.

Here's a common authentication flow using JWT:

1. **Login Request:** The user enters their credentials (e.g., email and password) on the client-side and sends them to the server via a POST request to a /login endpoint.
2. **Verification:** The server checks the credentials against the user records stored in the database.
3. **Token Generation:** If the credentials are valid, the server generates a JWT. This token contains a "payload" of data (like the user_id and role) and is digitally signed with a secret key known only to the server.
4. **Token Response:** The server sends this JWT back to the client in the response.
5. **Token Storage:** The client stores the JWT securely (e.g., in localStorage or an HTTP-only cookie).
6. **Authenticated Requests:** For every subsequent request to a protected endpoint, the client must include the JWT in the Authorization header, typically using the Bearer schema (e.g., Authorization: Bearer <your_jwt_here>).
7. **Server Protection:** A middleware on the server intercepts these requests, checks the Authorization header for a valid, unexpired token, and verifies its signature using the secret key. If the token is valid, the request is allowed to proceed to its handler; otherwise, the server sends back an "unauthorized" error.

This process ensures that only users with a valid token can access protected resources.

Resources

- **JWT.io Introduction:** The official and essential guide to JSON Web Tokens.
- **jsonwebtoken (npm package):** The most popular library for creating and verifying JWTs in Node.js.

Your Task

It's time to build a complete back-end application for a simple **Blog API**.

1. Basic Node.js Server:

- Initialize a new Node.js project (npm init -y).
- Using only the built-in http module, create a server that listens on a port and sends back a simple JSON response (e.g., { "message": "Hello, World!" }).
- Use Postman to send a GET request to your server and verify you get the response.

2. Express Setup and Routing:

- Install Express (npm install express).
- Refactor your server to use Express.
- Create five different routes:
 - GET /posts (to get all posts)
 - GET /posts/:id (to get a single post)
 - POST /posts (to create a new post)
 - PUT /posts/:id (to update a post)
 - DELETE /posts/:id (to delete a post)
- For now, have each route send back a simple text response confirming which route was hit.
- Implement a logging middleware that prints the HTTP method and URL of every incoming request to the console.

3. Database Connection:

- Set up a local PostgreSQL database with a posts table (id, title, content, created_at).
- Install node-postgres (npm install pg).
- Connect your Express server to the database.

4. Implement CRUD Operations:

- Update your five routes to perform the corresponding CRUD operation on the posts table in your database.
- **GET /posts:** Should query the database and return all posts as a JSON array.
- **POST /posts:** Should take a title and content from the request body, insert a new post into the database, and return the newly created post.
- Use Postman to thoroughly test each of these endpoints.

5. JWT Authentication:

- Add a users table to your database (id, email, password_hash).
- Install bcryptjs (npm install bcryptjs).
- Install any jwt library
- Create two new routes:
 - POST /register: Takes an email and password, hashes the password with bcryptjs, and saves the new user to the database.
 - POST /login: Takes an email and password, verifies them, and if they are

correct, returns a JWT.

- Create an authentication middleware that verifies the JWT from the Authorization header.
- Protect the routes for creating, updating, and deleting posts (POST /posts, PUT /posts/:id, DELETE /posts/:id) using this middleware. Requests to these routes without a valid token should be rejected.

6. **Integrate with Gemini API (Chatbot Feature):**

- Create a new protected POST route: /chat.
- This route should accept a JSON body with a prompt field (e.g., { "prompt": "Explain what a RESTful API is in simple terms." }).
- Inside the route handler, make a fetch request to the Gemini API with the user's prompt.
- Parse the response from Gemini and send the generated text back to the client as a JSON response (e.g., { "response": "A RESTful API is like..." }).
- Ensure this route is protected by your JWT authentication middleware so only logged-in users can interact with the chatbot.